

SUMMITY

Mountain Safety System — Sistem Perizinan & Pelacakan Pendakian Gunung Slamet

Materi Sidang · Dokumentasi Teknis · September 2026 · github.com/DESFIANFM/SUMMITY

1. Ringkasan Sistem

SUMMITY adalah aplikasi web progresif (PWA) untuk mengelola **SIMAKSI** (Surat Izin Masuk Kawasan Konservasi) dan melacak posisi pendaki di jalur Gunung Slamet via Basecamp Bambangan.

Masalah yang dipecahkan: pencatatan pendakian yang masih manual membuat petugas sulit mengetahui **siapa yang masih berada di atas gunung**. Data itu krusial saat terjadi keadaan darurat atau operasi pencarian.

Tiga masalah utama dan solusinya

Masalah	Pendekatan dalam sistem
Pendataan manual, rawan hilang	SIMAKSI digital dengan verifikasi petugas dan riwayat status
Tidak ada sinyal di gunung	Arsitektur <i>offline-first</i> : data disimpan lokal di perangkat, disinkronkan otomatis saat sinyal kembali
Tidak tahu siapa yang belum turun	Pelacakan berbasis QR per pos + mekanisme lapor kepulangan

Dua peran pengguna

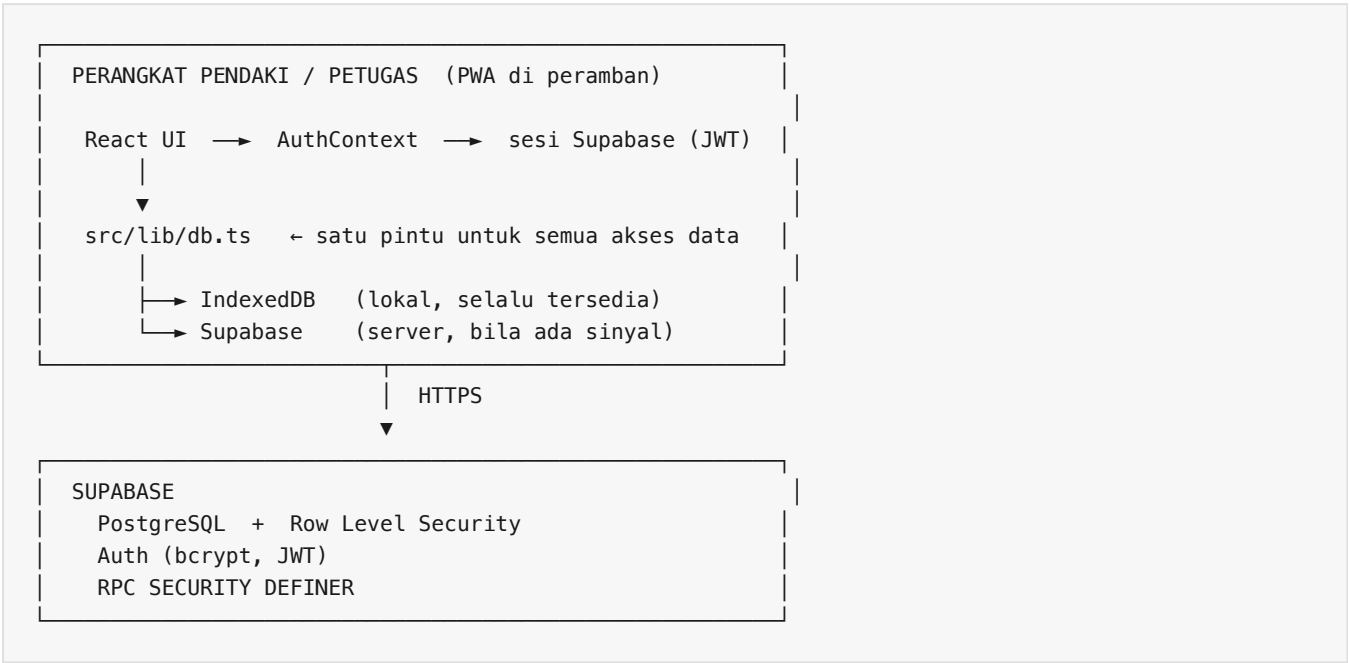
Peran	Kemampuan
Pendaki USER	Mendaftar akun, mengajukan SIMAKSI (sendiri atau rombongan), memindai QR pos untuk mencatat progres, melihat tiket dan riwayat perjalanan
Petugas ADMIN	Memverifikasi (menyetujui/menolak) SIMAKSI, memantau sebaran pendaki per pos, melihat log aktivitas dan detail identitas, memindai QR lapor kepulangan

2. Teknologi yang Dipakai

Lapisan	Teknologi	Alasan pemilihan
Antarmuka	React 19 + TypeScript	Komponen dapat dipakai ulang; TypeScript menangkap kesalahan tipe sebelum aplikasi dijalankan
Build tool	Vite 6	Build produksi cepat (± 7 detik), hot reload saat pengembangan
Gaya	Tailwind CSS 4	Utility-first, konsisten tanpa menulis file CSS terpisah
Routing	React Router 7	Navigasi sisi klien, mendukung rute terproteksi berbasis peran
Basis data server	Supabase (PostgreSQL)	PostgreSQL terkelola, sudah menyediakan autentikasi dan <i>Row Level Security</i>

Basis data lokal	IndexedDB (pustaka <code>idb</code>)	Penyimpanan di peramban berkapasitas besar; syarat mutlak agar aplikasi jalan tanpa sinyal
Autentikasi	Supabase Auth	Password di-hash bcrypt, sesi berbasis JWT, reset password bawaan
PWA	vite-plugin-pwa (Workbox)	Dapat dipasang di layar utama, aset di-cache, tetap terbuka saat offline
Pemindai QR	html5-qrcode	Membaca QR lewat kamera perangkat langsung dari peramban
Peta	Leaflet + React-Leaflet	Peta jalur pendakian; ubin peta dapat di-cache untuk dipakai offline
Animasi	Motion	Transisi antar panel pada antarmuka
Pengujian	Vitest + Testing Library	84 pengujian unit dan integrasi
Hosting	Vercel	Deploy otomatis dari branch <code>main</code> , domain <code>www.summity.id</code>

3. Arsitektur



Prinsip penting: satu pintu akses data

Seluruh komponen antarmuka tidak pernah memanggil Supabase secara langsung. Semuanya melewati `src/lib/db.ts`. Setiap fungsi di sana memakai pola yang sama:

```
if (ada koneksi && Supabase tersedia) {
  ambil dari Supabase          → data terbaru
} else {
  ambil dari IndexedDB          → data lokal, mungkin tertinggal
}
```

Keuntungannya: logika online/offline terpusat di satu berkas, sehingga komponen antarmuka tidak perlu tahu sedang online atau tidak.

4. Skema Basis Data

Delapan tabel di skema `public`. Kolom yang dicetak tebal adalah kunci relasi antar tabel.

Tabel	Kolom
users Identitas pendaki & petugas	<code>id</code> (UUID, sama dengan <code>auth.users.id</code>), <code>id_pendaki</code> (nomor tampilan, mis. 202606180001), <code>name</code> , <code>email</code> , <code>username</code> , <code>role</code> (USER/ADMIN), <code>nik</code> , <code>phone</code> , <code>emergency_phone</code> , <code>gender</code> , <code>citizenship</code> , <code>identity_type</code> , <code>weight</code> , <code>height</code> , <code>province</code> , <code>city</code> , <code>district</code> , <code>subdistrict</code> , <code>address</code> , <code>created_at</code> , <code>updated_at</code>
simaksi Pengajuan izin pendakian	<code>id</code> , <code>kode_simaksi</code> (mis. SMK-202606-0023), <code>ketua_user_id</code> → <code>users</code> , <code>gunung_id</code> , <code>tanggal_naik</code> , <code>tanggal_turun</code> , <code>total_anggota</code> , <code>status</code> , <code>catatan_verifikator</code> , <code>approved_by</code> , <code>approved_at</code> , <code>estimasi_checkin</code> , <code>estimasi_checkout</code> , <code>checkin_at</code> , <code>checkout_at</code> , <code>last_location_lat</code> , <code>last_location_lng</code>

simaksi_anggota Anggota rombongan	id, simaksi_id → simaksi, user_id → users, created_at
tracking_history Riwayat pemindaian pos	id, ticket_id → tickets, user_id → users, pos_id → pos, scanned_at , device_id , is_offline , synced_at , validation_status , notes , created_by
tickets Tiket pendakian	id, user_id → users, mountain_name , date , end_date , status , qr_code
pos Titik checkpoint jalur	id (0-10), name , elevation , distance_from_base , latitude , longitude , description
mandatory_gear Daftar perlengkapan wajib	id, kode , nama_barang , kategori , aktif
simaksi_status_history Jejak perubahan status	Tersedia untuk audit; belum dipakai aplikasi

Siklus status SIMAKSI



Status	Arti
pending	Diajukan pendaki, menunggu verifikasi petugas
approved	Disetujui; kode_simaksi terbit dan tiket dibuat
rejected	Ditolak; alasan tersimpan di catatan_verifikator
checkin / checkout	Sedang mendaki / sudah lapor kepulangan
complete	Perjalanan selesai dan tercatat tuntas

5. Model Keamanan

Bagian ini paling mungkin ditanyakan penguji, karena menyangkut perlindungan data pribadi pendaki (NIK, alamat, nomor telepon).

Dua lapis pertahanan

Lapis 1 — Autentikasi (Supabase Auth). Password tidak pernah disimpan sebagai teks biasa maupun dikirim ke peramban. Supabase menyimpannya ter-*hash* dengan bcrypt di skema `auth` yang terpisah. Setelah login, klien memegang JWT berisi identitas pengguna.

Lapis 2 — Otorisasi (Row Level Security). PostgreSQL memeriksa setiap baris terhadap aturan yang ditulis di basis data. Aturan ini berlaku **di sisi server**, sehingga tidak bisa dilewati dengan memodifikasi kode JavaScript di peramban.

Konsep kunci untuk dijelaskan saat sidang:

`anon key` yang tertanam di JavaScript frontend memang bersifat publik — siapa pun bisa membacanya lewat DevTools. Yang melindungi data **bukan kerahasiaan kunci itu**, melainkan Row Level Security. Tanpa RLS yang benar, kunci publik itu membuka seluruh basis data.

Aturan akses per tabel

Tabel	Pendaki	Petugas
<code>users</code>	Hanya barisnya sendiri (<code>id = auth.uid()</code>)	Semua baris
<code>simaksi</code>	Miliknya sebagai ketua atau anggota. Tidak boleh mengubah status	Semua baris, dan satu-satunya yang boleh mengubah status
<code>simaksi_anggota</code>	Barisnya sendiri, atau anggota rombongan yang ia pimpin	Semua baris
<code>tracking_history</code>	Riwayat dirinya sendiri; hanya bisa menambah atas namanya sendiri	Semua baris

Hasil pengujian nyata (lewat sesi login sungguhan)

Tabel	Pendaki	Petugas	Belum login
<code>users</code>	1 dari 21	21 dari 21	ditolak
<code>simaksi</code>	4 dari 12	12 dari 12	ditolak
<code>simaksi_anggota</code>	3 dari 7	7 dari 7	ditolak
<code>tracking_history</code>	7 dari 132	132 dari 132	ditolak

Fungsi RPC SECURITY DEFINER

Sebagian kebutuhan tidak bisa dipenuhi RLS biasa. Misalnya, ketua rombongan perlu mencari pendaki lain dari ID Pendaki-nya, padahal RLS melarang membaca baris orang lain. Solusinya fungsi `SECURITY DEFINER` : fungsi berjalan dengan hak pemiliknya sehingga melewati RLS, tetapi **hasil yang dikembalikan dibatasi secara ketat**.

Fungsi	Kegunaan dan batasannya
<code>get_login_email()</code>	Menerjemahkan username ke email untuk login. Hanya mengembalikan satu kolom email
<code>is_admin()</code>	Memeriksa apakah pemanggil berperan ADMIN. Dipakai di hampir semua policy
<code>find_pendaki()</code>	Mencari pendaki untuk ditambahkan ke rombongan. Hanya mengembalikan <code>id</code> , <code>id_pendaki</code> , dan <code>name</code> — tidak pernah NIK, telepon, atau alamat
<code>get_simaksi_members()</code>	Daftar anggota rombongan. Memeriksa otorisasi di dalam fungsi; pemanggil yang tidak berhak menerima daftar kosong
<code>is_simaksi_ketua()</code> <code>is_simaksi_member()</code>	Pemeriksaan lintas tabel di dalam policy. Memakai fungsi mencegah <i>rekursi tak hingga</i> antar policy

Perbandingan sebelum dan sesudah penerapan

Kondisi awal	Kondisi sekarang
Password tersimpan sebagai teks biasa di tabel <code>users</code>	Ter-hash bcrypt di <code>auth.users</code>
Password petugas ditulis langsung di kode frontend	Akun petugas sungguhan; peran ditentukan kolom <code>role</code> di basis data
Siapa pun dapat membaca NIK, alamat, dan telepon seluruh pendaki	Pendaki hanya dapat membaca datanya sendiri
Siapa pun dapat menyetujui SIMAKSI miliknya sendiri	Hanya petugas yang dapat mengubah status
Riwayat pelacakan dapat diubah dan dihapus	Hanya dapat ditambah atas nama sendiri

6. Alur Utama Sistem

6.1 Pendaftaran akun

```
Formulir 3 tahap (akun → data diri → alamat)
└─> supabase.auth.signUp()      buat akun, password di-hash
    └─> INSERT ke tabel users    id = id akun Auth ← kunci agar RLS bekerja
        └─> trigger database    menerbitkan id_pendaki otomatis
```

Penyamaan `users.id` dengan `auth.users.id` adalah fondasi seluruh model keamanan — semua policy bertumpu pada perbandingan `id = auth.uid()`.

6.2 Login

```
Pengguna memasukkan username + password
└─> RPC get_login_email(username) → email
    └─> signInWithPassword(email, password) → sesi JWT
        └─> ambil profil dari users → peran dibaca dari kolom role
```

Peran **tidak ditentukan di frontend**. Aplikasi membaca kolom `role` dari basis data, sehingga pengguna tidak dapat mengaku sebagai petugas.

6.3 Pengajuan SIMAKSI

```
Pendaki mengisi tanggal, anggota rombongan, ceklis perlengkapan wajib
└─> simpan ke IndexedDB          status pending, synced = false
    └─> bila ada sinyal: kirim ke Supabase
        ├── INSERT simaksi       status wajib 'pending'
        └── INSERT simaksi_anggota lewat RPC find_pendaki
```

6.4 Verifikasi oleh petugas

```
Dashboard petugas menampilkan daftar pengajuan pending
└─> SETUJUI → status approved, kode_simaksi terbit, tiket dibuat
└─> TOLAK   → status rejected + catatan_verifikator
```

6.5 Pelacakan di jalur

```
Pendaki memindai QR yang terpasang di tiap pos
└─> INSERT tracking_history (user_id, pos_id, waktu)
    └─> offline: disimpan lokal, dikirim otomatis saat sinyal kembali
```

Dashboard petugas menghitung sebaran pendaki per pos dari pemindaian terakhir tiap tiket, sekaligus membedakan arah **naik** dan **turun** — arah turun dikenali setelah pendaki tercatat mencapai puncak.

6.6 Laporan kepulangan

```
Pendaki menampilkan QR SUMMITY-RETURN-{id} dari halaman Tiket
└─ petugas memindai dengan Scanner Petugas
    └─ completeSimaksi() status → complete
        └─ catatan checkout dibuat untuk seluruh anggota
```

6.7 Lupa password

```
Halaman login → "Lupa Password?" → masukkan email
└─ resetPasswordForEmail(email, redirectTo: /reset-password)
    └─ email berisi tautan sekali pakai
        └─ halaman /reset-password → sesi recovery → password baru
```

Pesan keberhasilan sengaja dibuat sama untuk email terdaftar maupun tidak, agar halaman ini tidak dapat dipakai menebak email mana yang ada di sistem.

7. Sistem QR Code

Format	Lokasi	Pemindai	Fungsi
SUMMITY-POS-{0..10}	Dicetak & dipasang di tiap pos	Pendaki	Mencatat progres perjalanan
SUMMITY-RETURN-{id}	Layar HP pendaki	Petugas	Lapor kepulangan
SUMMITY-SIMAKSI-{id}	Layar HP pendaki	Petugas	Identitas tiket pendakian

Jalur Gunung Slamet via Bambangan dimodelkan sebagai **11 titik**: Basecamp Bambangan (1.502 mdpl) sampai Puncak Slamet (3.428 mdpl), sejauh 10,5 km.

8. Struktur Kode

Berkas	Baris	Tanggung jawab
<code>src/lib/db.ts</code>	1887	Seluruh akses data; logika online/offline dan sinkronisasi
<code>src/lib/auth.ts</code>	221	Login, pendaftaran, reset password, pengambilan profil
<code>src/context/AuthContext.tsx</code>	138	Status sesi global; pemulihan sesi saat aplikasi dibuka
<code>src/pages/user/Register.tsx</code>	1276	Formulir pendaftaran tiga tahap beserta validasinya
<code>src/pages/admin/Dashboard.tsx</code>	1163	Pemantauan pendaki, verifikasi SIMAKSI, log aktivitas, pencarian pendaki
<code>src/pages/Login.tsx</code>	456	Login pendaki dan petugas, panel lupa password
<code>src/pages/user/SimaksiSubmit.tsx</code>	471	Pengajuan SIMAKSI beserta ceklis perlengkapan
<code>src/components/GPSMap.tsx</code>	447	Peta jalur pendakian (Leaflet)
<code>src/pages/user/Scanner.tsx</code>	301	Pemindai QR pos untuk pendaki
<code>src/pages/admin/Scanner.tsx</code>	224	Pemindai QR lapor kepulangan untuk petugas
<code>src/pages/ResetPassword.tsx</code>	145	Halaman pembuatan password baru
<code>supabase/migrations/</code>	6 berkas	Skema keamanan: RLS, fungsi RPC, dan trigger

Fungsi kunci di `db.ts`

Fungsi	Kegunaan
<code>saveSimaksi()</code>	Menyimpan pengajuan ke lokal lalu mencoba menyinkronkan
<code>approveSimaksi()</code> / <code>rejectSimaksi()</code>	Keputusan verifikasi petugas
<code>completeSimaksi()</code>	Menuntaskan perjalanan saat lapor kepulangan
<code>insertTrackingHistory()</code>	Mencatat pemindaian pos
<code>getAllTrackingHistory()</code>	Log aktivitas untuk dashboard petugas
<code>getHikerStatuses()</code>	Daftar pendaki beserta status perjalanannya
<code>getSimaksiDetail()</code>	Detail lengkap satu SIMAKSI beserta identitas anggotanya
<code>syncAllUnsyncedData()</code>	Menyinkronkan seluruh data tertunda saat sinyal kembali

9. Mekanisme Offline-First

Bagian ini merupakan pembeda utama sistem, karena di gunung umumnya tidak ada sinyal.

Mekanisme	Cara kerja
Penyimpanan ganda	Setiap penulisan masuk ke IndexedDB lebih dulu, ditandai <code>synced: false</code>
Sinkronisasi otomatis	Berjalan saat peristiwa <code>online</code> terdeteksi, dan diperiksa ulang berkala tiap 10 detik
Cache aset	Service worker menyimpan HTML, JS, dan CSS sehingga aplikasi tetap terbuka tanpa jaringan
Cache peta	Ubin peta OpenStreetMap disimpan hingga 30 hari untuk dipakai di jalur
Sesi bertahan	Sesi login tersimpan di perangkat; setelah login online pertama, aplikasi tetap dapat dipakai offline

Batasan yang perlu diakui jujur saat sidang: login *pertama kali* tetap memerlukan koneksi internet. Ini konsekuensi dari memindahkan autentikasi ke server — sebelumnya login offline dimungkinkan karena password tersimpan sebagai teks biasa di perangkat, dan justru itulah celah keamanan yang diperbaiki.

10. Pengujian

84 pengujian pada 9 berkas, seluruhnya lulus. Dijalankan dengan `npm test`.

Berkas uji	Cakupan
<code>db.simaksi.test.ts</code>	Alur SIMAKSI: pengajuan, persetujuan, penolakan, penuntasan
<code>db.integration.test.ts</code>	Mode offline dengan IndexedDB tiruan
<code>db.sync.test.ts</code>	Sinkronisasi data tertunda
<code>AuthContext.test.tsx</code>	Pemulihan sesi, cache profil offline, login dan logout
<code>ProtectedRoute.test.tsx</code>	Pembatasan akses halaman berdasarkan peran
<code>formatters.test.ts</code> , <code>mockData.test.ts</code> , <code>tileCache.test.ts</code> , <code>db.helpers.test.ts</code>	Fungsi pembantu: format tanggal, estimasi waktu tempuh, cache ubin peta

11. Antisipasi Pertanyaan Penguji

Mengapa memakai Supabase, bukan membangun backend sendiri?

Supabase menyediakan PostgreSQL, autentikasi, dan otorisasi tingkat baris dalam satu paket terkelola. Membangun sendiri berarti mengimplementasikan ulang hashing password, manajemen sesi, dan kontrol akses — bagian yang paling rawan salah dan berisiko tinggi bila keliru.

Bagaimana data pendaki dilindungi?

Dua lapis. Autentikasi memastikan pengguna adalah dirinya (password ter-hash bcrypt, sesi JWT). Otorisasi Row Level Security memastikan ia hanya melihat datanya sendiri — aturan ini dievaluasi PostgreSQL di server, sehingga tidak dapat dilewati dengan mengubah kode di peramban.

Kunci Supabase terlihat di kode frontend. Bukankah itu berbahaya?

Kunci `anon` memang dirancang untuk publik. Yang melindungi data adalah Row Level Security, bukan kerahasiaan kunci. Kunci `service_role` yang benar-benar rahasia tidak pernah dimasukkan ke frontend.

Bagaimana kalau pendaki mengubah kode JavaScript untuk menyetujui SIMAKSI-nya sendiri?

Tidak berhasil. Perubahan status SIMAKSI dijaga policy `USING (is_admin())` yang dievaluasi PostgreSQL. Kode frontend hanya mengirim permintaan; keputusan akhir ada di basis data.

Bagaimana aplikasi bekerja tanpa sinyal di gunung?

Data ditulis ke IndexedDB di perangkat lebih dulu dan ditandai belum tersinkron. Saat sinyal kembali, sinkronisasi berjalan otomatis. Service worker menjaga aplikasi tetap dapat dibuka, dan ubin peta di-cache untuk dipakai di jalur.

Bagaimana sistem mengetahui pendaki sudah turun atau belum?

Dari dua sumber: riwayat pemindaian QR per pos, dan status SIMAKSI. Pendaki berstatus `approved` atau `checkin` dianggap masih di atas; `checkout` atau `complete` berarti sudah lapor pulang. Dashboard petugas menyediakan pencarian nama beserta filter status ini.

Apa yang terjadi jika dua pendaki memakai NIK yang sama?

Ditolak basis data melalui *unique index* pada kolom `nik`, `username`, dan `email`. Pemeriksaan di frontend saja tidak cukup karena dapat dilewati.

Apa keterbatasan sistem saat ini?

Tiga hal yang perlu disampaikan jujur: login pertama kali memerlukan internet; pengiriman email bergantung pada kuota penyedia SMTP; dan pemindaian pos masih mengandalkan kejujuran pendaki karena QR pos dapat difoto — verifikasi lokasi berbasis GPS dapat menjadi pengembangan lanjutan.

Apa rencana pengembangan selanjutnya?

Verifikasi GPS saat pemindaian pos, notifikasi otomatis bila pendaki melewati estimasi waktu turun, dukungan banyak gunung, serta laporan statistik kunjungan untuk pengelola.

12. Ringkasan Angka

Aspek	Nilai
-------	-------

Jumlah berkas sumber (tanpa berkas uji)	25
Total baris kode	± 8.500
Tabel basis data	8
Fungsi RPC keamanan	6
Berkas migrasi SQL	6
Pengujian otomatis	84 (seluruhnya lulus)
Titik pos pada jalur	11 (Basecamp – Puncak)
Ukuran bundle produksi	1,43 MB (407 KB setelah kompresi gzip)
Waktu build	± 7 detik